Session Management and JWT

Laboratory for the class “Offensive Security”
(01NWLWQ, 01NWLUV, 01NWLUW, 01NWLWR)
Politecnico di Torino – AY 2025/26
Prof. Cataldo Basile

prepared by:
Stefano Alberto (stefano.alberto@polito.it)

v. 1.0.0 (13/3/2026)

Contents

1 Introduction

The purpose of this laboratory is to explore how web applications manage authentication and sessions, and to identify common vulnerabilities in these mechanisms. In particular, we focus on cookie-based sessions and JSON Web Tokens (JWT).

Authentication

Almost every web application needs to deal with two fundamental problems:

1. **How to authenticate** a user — typically through a username and password (ideally not stored in plaintext), and possibly a second factor (2FA).
2. **How to keep a user authenticated** across multiple requests, so they do not have to log in again on every page load.

The first problem must be implemented carefully to prevent attacks such as brute-force login attempts or 2FA bypass. The second problem is the main focus of this lab.

HTTP is stateless

HTTP is a **stateless** protocol: each request is independent, and the server has no built-in way to link two requests as coming from the same client. To maintain state — such as "this user is logged in" — web applications use **cookies**.

A cookie is a small piece of data that the server sends to the browser via the `Set-Cookie` response header. The browser then automatically includes it in every subsequent request to the same domain via the `Cookie` header. This turns HTTP into a stateful protocol.

The key question is: *how do we store authentication information in a cookie securely?* If done incorrectly, an attacker can tamper with the cookie to impersonate another user.

Server-side sessions

The traditional approach is to store all session data **on the server** and give the client only a random, opaque **session identifier**.

For example, PHP's `session_start()` generates a random string like `7466e31642a819b596200676443685e8` and stores it in a cookie (typically named `PHPSESSID`). On the server, a file named `sess_7466e31642a819b59620067644368` holds the actual session data (e.g. the username, role, preferences).

This approach is secure **if and only if**:

- The session ID is generated using a **cryptographically secure** random number generator,
- The ID has **enough entropy** (length and randomness) to make brute-forcing infeasible.

If the session ID space is too small or predictable, an attacker can try random values until they find a valid session belonging to another user.

> NOTA
>
> A common pitfall is using sequential integers, timestamps, or other predictable values as session identifiers. Even encoding them in Base64 does not add security — Base64 is an **encoding**, not encryption. It is trivially reversible and provides no confidentiality or integrity.

The main drawback of server-side sessions is scalability: all servers must have access to the session store. When scaling horizontally across multiple servers or data centers, a centralized session store (database, Redis) is required, which can become a bottleneck.

Client-side sessions and JWT

An alternative is to store session data directly **on the client**, in a way that prevents tampering. There are two main strategies:

- **Signing** the data, so the server can verify it has not been modified (JWT),
- **Encrypting** the data, so the client cannot read or modify it (JWE).

JSON Web Tokens (JWT) are the most common implementation of signed client-side sessions. A JWT consists of three Base64url-encoded parts separated by dots:

```
header.payload.signature
```

- The **header** specifies the token type and signing algorithm (e.g. `{"alg":"HS256","typ":"JWT"}`),
- The **payload** contains the claims — the actual session data (e.g. `{"sub":"user123","admin":false}`),
- The **signature** is computed over the header and payload using a secret key, ensuring integrity.

Because the header and payload are only Base64url-encoded (not encrypted), anyone can **read** them. The signature guarantees that the data has not been **modified** — but only if the server actually validates it.

> NOTA
>
> The security of JWT relies entirely on **signature verification**. If the server does not validate the signature, or accepts tokens with the algorithm set to `none`, an attacker can forge arbitrary tokens. Always use well-tested libraries to handle JWT, and never implement signature verification manually.

The main advantages of JWT over server-side sessions are:

- No server-side state is required — any server with the signing key can verify the token independently,
- Easy to scale horizontally across multiple servers and data centers.

The main disadvantages are:

- Tokens cannot be easily **invalidated** before expiration (revoking a specific session requires adding server-side state, defeating the purpose),
- Implementation errors are more dangerous — a bug in signature verification can compromise all sessions at once.

For a detailed overview of JWT attacks, see:

https://portswigger.net/web-security/jwt

For more on authentication vulnerabilities in general:

https://portswigger.net/web-security/authentication

2 Exercises

This section includes seven exercises on session management and JWT vulnerabilities in web applications. The first two are solved step by step; the next three are core exercises with less hand-holding; the last two are optional and cover more advanced topics.

Exercise 1 – Cookie Monster Army

Purpose:	Impersonate the admin by tampering with the session cookie.
Suggested tools:	Browser DevTools (Application/Storage tab), Burp Suite.
Source:	OliCyber Training – https://training.olicyber.it/challenges#challenge-51

The "Cookie Monster Army" recruitment portal uses a cookie-based session mechanism. Your task is to understand how the session cookie works and exploit it to impersonate the admin. Follow the steps below.

Explore the application

Open http://cma.challs.olicyber.it in your browser. The page presents a login form and an "Arruolati" (enlist) button to register a new account. Create an account and log in.

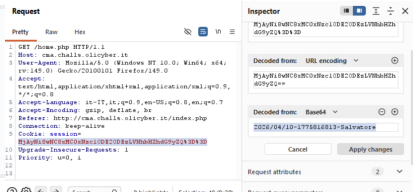
After logging in, you land on a welcome page. Does it show you the flag? What information does the page display about your identity?

→ No,It shows information about the cookie sessionID

Inspect the session cookie

Open the browser DevTools (F12), go to the **Application** tab (Chrome) or **Storage** tab (Firefox), and inspect the cookies set for `cma.challs.olicyber.it` after logging in.

You should find a cookie that was not present before authentication. What is it called? Does the value look like a random session ID, or does it have some recognizable structure?

→ 

Decode the cookie

The cookie value ends with = or consists only of alphanumeric characters, +, and / — a strong hint that it is Base64-encoded. Copy it and decode it using the Burp Suite Decoder tab, the browser console (`atob("...")`), or the command line (`echo "<value>" | base64 -d`).

What fields does the decoded string contain? Can you spot your username in it?

→ It contains the following info: 2026/04/10-1775816813-Salvatore

NOTA

Base64 is an **encoding**, not encryption. It is a fully reversible transformation designed for transporting binary data in text-based protocols. It provides **no confidentiality** and **no integrity** — anyone can decode it, modify the contents, and re-encode it. Never use Base64 alone to protect sensitive data.

Forge the admin cookie

Now that you know the cookie format, replace the username field with `admin`, keeping the other fields unchanged. Encode the modified string back to Base64 (e.g. with `echo -n "..." | base64`) and overwrite the `session` cookie in DevTools or Burp Suite.

Reload `home.php`. Does the server accept your forged cookie? What do you see now?

→ flag{c00ki3_4rmy_will_c0nqu3r_th3_w0rld!}

Reflection

The cookie contained your username in plaintext (only Base64-encoded) and the server accepted a modified value without complaint. Why is this session mechanism fundamentally broken? What should the developer do to prevent cookie forgery?

→ Was broken because the server does not do a "double-check" to verity the information, moreover the cookie is not protected from tampering (digital signature) and store session informatio in client-side.
The developer should protect the cookie from tampering using digital signature, avoid to store authN information in client-side, and implement a check mechanism in server-side.

Exercise 2 – A TOO small reminder...

Purpose:	Hijack the admin's session by brute-forcing a weak session identifier.
Suggested tools:	Burp Suite (Repeater, Intruder), Python/Bash.
Source:	OliCyber Training – https://training.olicyber.it/challenges#challenge-36

This application exposes a JSON API with five endpoints: `/` (description), `/register`, `/login`, `/logout`, and `/admin`. Your task is to register, log in, understand how the session works, and hijack the admin's session. Follow the steps below.

Explore the API and register

Open http://too-small-reminder.challs.olicyber.it in your browser (or send a `GET /` through Burp Repeater). The JSON response lists every endpoint with its accepted method and parameters. Use this information to register a new account and log in.

After a successful login, inspect the response headers in Burp. What cookie does the server set? What does the value look like — is it a long random token, or something simpler?

Analyze the session ID

Log out and log in again a few times, or register multiple accounts. Compare the session IDs you receive each time.

How large is the session ID space? Could an attacker realistically try them all?

> NOTA
>
> A secure session identifier must be generated with a **cryptographically secure pseudo-random number generator** (CSPRNG) and must have **enough entropy** to make guessing infeasible. OWASP recommends at least 128 bits of entropy. A session ID below 10 000 has fewer than 14 bits of entropy — trivially brute-forceable.

Access the admin endpoint

Send a `GET /admin` request using Burp Repeater with your own session cookie. Then try the same request without any cookie, and with a made-up cookie value. Compare the HTTP status codes and response bodies across the three cases.

You should observe distinct responses depending on whether the session is missing, belongs to a regular user, or belongs to the admin. Think about which property (status code, body length, content) you can use to tell them apart automatically.

Brute-force the admin session

The admin has an active session somewhere in the ID space you identified. Enumerate all possible values and look for the response that stands out.

Option A — Burp Suite Intruder:

1. Send the `GET /admin` request (with the session cookie) to Intruder.
2. In the **Positions** tab, mark only the cookie value as the injection point.
3. In the **Payloads** tab, use a **Numbers** payload covering the full ID range with step 1.
4. Start the attack. Sort results by **Status code** or **Length** to spot the anomaly.

Option B — Write a script: Write a short Python (or Bash) script that iterates over the same range, sends a request for each ID, and prints any response whose status code differs from the majority. The Python `requests` library is a good fit.

Capture the flag

Once you find the anomalous session ID, fetch the `/admin` response for that ID. What is the flag?

Reflection

Why is this approach feasible here but not on a well-designed application?

→

Exercise 3 – Power Cookie

Purpose:	Gain admin access by manipulating a cookie that controls authorization.
Suggested tools:	Browser DevTools, Burp Suite.
Source:	picoCTF – https://play.picoctf.org/practice/challenge/288

The "Online Gradebook" application lets you continue as a guest. Your task is to gain admin privileges and retrieve the flag.

Enter as guest and observe the behavior

Open the URL in your browser. The homepage shows an **"Online Gradebook"** heading with a single button: **"Continue as guest"**. Click the button. You are redirected to `/check.php`, which displays a message denying you access.

Before moving on, look at the page source or the JavaScript file loaded on the homepage. What does the button actually do before navigating to `/check.php`?

→

Inspect the cookie and escalate

Open DevTools and check the cookies set for this site (Application tab → Cookies, or use Burp Suite to inspect the request to `/check.php`). You should find a cookie whose name and value make the vulnerability immediately obvious.

Modify the cookie value to grant yourself admin privileges, then reload `/check.php`.

→

NOTA

This is a direct example of why **authorization decisions must never rely on client-controlled data** without server-side verification. A cookie value can be freely modified by the user. The server must maintain its own record of the user's role and check it on every request.

Exercise 4 – Bibvault 1

Purpose:	Exploit a JWT implementation that does not validate the token signature.
Suggested tools:	Burp Suite, https://jwt.io (or Burp's JWT Editor extension).
Hints:	Log in and examine the token you receive. What format is it in? Decode all three parts. What happens if you modify the payload and send it back without changing the signature?
Source:	OliCyber Training – https://training.olicyber.it/challenges#challenge-305

This application uses JWT for session management. After logging in, you receive a token. Analyze its structure and determine whether the server properly validates the signature.

What algorithm does the header claim? What claims are in the payload?

Can you modify the payload to escalate your privileges? Does the server reject the modified token?

→

NOTA

This is one of the most critical JWT vulnerabilities: the server **ignores the signature entirely**. The token might as well be unsigned. This typically happens when developers decode the JWT to read the claims but forget (or skip) the verification step. Always use a library that verifies the signature as part of the decoding process.

Exercise 5 – JWT bypass via flawed signature verification

Purpose:	Bypass JWT authentication by exploiting the `alg: none` vulnerability.
Suggested tools:	Burp Suite (with JWT Editor extension, or manual Base64 editing).
Hints:	The JWT header specifies which algorithm was used to sign the token. What happens if you tell the server that no algorithm was used?
Source:	PortSwigger Web Security Academy – https://portswigger.net/web-security/jwt/lab-jwt-authentication-bypass-via-flawed-signature-verification

This is a guided lab from PortSwigger. The application uses JWT-based sessions, but the server's signature verification can be bypassed.

Log in with the provided credentials and examine the JWT. Modify the token so that the server accepts it without a valid signature. Access the admin panel and delete the target user to solve the lab.

→

NOTA

The `alg: none` attack works when the server trusts the algorithm specified in the token header instead of enforcing a fixed algorithm on the server side. A secure implementation must **never** accept `none` as a valid algorithm in production, and should use an allowlist of accepted algorithms rather than reading the algorithm from the token itself.

Exercise 6 (Optional) – JWT bypass via weak signing key

Purpose:	Crack a weak HMAC signing key and forge a valid JWT.
Suggested tools:	Burp Suite, hashcat (or John the Ripper).
Hints:	If the server uses HMAC (symmetric signing), the security depends entirely on the strength of the secret key. What if the key is a common word or short string?
Source:	PortSwigger Web Security Academy – https://portswigger.net/web-security/jwt/lab-jwt-authentication-bypass-via-weak-signing-key

This is a PortSwigger guided lab. The application uses JWT with HMAC-SHA256 (`HS256`), but the signing key is weak and can be cracked offline.

Log in with the provided credentials and capture the JWT. Use an offline brute-force tool to recover the signing secret, then forge a new token to access the admin panel.

→

NOTA

This exercise introduces an important principle: **symmetric signing keys must be strong**. HMAC-SHA256 is a secure algorithm, but only if the key has sufficient entropy. A short or dictionary-based key can be cracked offline without any interaction with the server. Use randomly generated keys of at least 256 bits.

Exercise 7 (Optional) – 2FA simple bypass

Purpose:	Bypass two-factor authentication by skipping the verification step.
Suggested tools:	Burp Suite, Browser.
Hints:	After entering the username and password, you are redirected to a 2FA page. Is this step actually enforced, or can you navigate directly to an authenticated page?
Source:	PortSwigger Web Security Academy – https://portswigger.net/web-security/authentication/multi-factor/lab-2fa-simple-bypass

This is a PortSwigger guided lab on authentication. The application implements two-factor authentication, but the implementation has a flaw in how it enforces the 2FA step.

You have valid credentials for a user whose 2FA code you do not know. Can you access the account without completing the 2FA step?

→

NOTA

This is a **logic vulnerability** in the authentication flow, similar to those we studied in Lab 1. The application assumes users will follow the intended flow (password $\rightarrow$ 2FA $\rightarrow$ account), but never verifies that all steps were completed. A secure implementation must track the authentication state and require **all steps** to be completed before granting access.

3 Extras

This section lists additional resources that are not mandatory for this laboratory. They provide further practice on the topics covered in this lab and can be used for self-study.

PortSwigger Web Security Academy

PortSwigger provides an excellent set of free, guided labs on many web security topics. The following learning paths are directly relevant to this laboratory:

- **JWT attacks:** `https://portswigger.net/web-security/jwt`

  Includes additional labs beyond those covered in the exercises, such as:

  - JWT authentication bypass via `jwk` header injection,
  - JWT authentication bypass via `jku` header injection,
  - JWT authentication bypass via `kid` header path traversal.

- **Authentication vulnerabilities:** `https://portswigger.net/web-security/authentication`

  Covers a wide range of authentication flaws including brute-force attacks, password-based vulnerabilities, and multi-factor authentication bypasses.